

Practical Session 16: AI/ML Foundations for Financial Text

Hands-On: Lexicon Sentiment, BoW vs. Embeddings, and a Mini Attention Trace

Juan F. Imbet

Paris Dauphine – PSL University

June 20, 2026

Session Overview

Duration: 1 hour (50 min activities + 10 min debrief)

- ① **Recap** (5 min): the AI/ML/DL/LLM hierarchy; the attention formula.
- ② **Problem 1** (12 min): Loughran–McDonald dictionary sentiment on a filing snippet — by hand — and why a *general* lexicon misleads.
- ③ **Problem 2** (10 min): Show that mean BoW pooling is order-blind on a financial sentence; quantify the failure.
- ④ **Problem 3** (8 min): Trace a 3×3 self-attention step on a financial sequence (uses the recap formula).
- ⑤ **Case Study** (15 min): Python — dictionary vs. embedding sentiment on SEC snippets (`code/notebooks/16-ai-ml-finance-text/`).

Discussion (10 min)

When is a 1990s-vintage dictionary still the *right* tool, and when must you reach for an embedding or an LLM?

Recap: The Hierarchy and Where Text Fits

THE BIGGER PICTURE

AI $\supset$ ML $\supset$ DL $\supset$ LLM. **Statistical** AI (learn parameters from data) dominates finance over **symbolic** AI (hand-coded rules) because markets are data-rich — but symbolic rules return for compliance and interpretability.

Why text is hard (carry into the problems):

- **Ambiguous:** $\sim 3/4$ of general “negative” words are neutral/positive in filings $\Rightarrow$ use a *finance* lexicon.
- **Sparse:** 50k-word vocabulary $\gg$ observations.
- **Strategic:** disclosures written by lawyers — not unbiased signals.

Recap: Two Formulas You Will Use

Dictionary sentiment of a document:

$$\text{Sentiment} = \frac{\# \text{positive words} - \# \text{negative words}}{\# \text{total words}}.$$

Mean bag-of-words embedding (order-blind by construction):

$$\bar{\varphi}(d) = \frac{1}{n} \sum_{i=1}^n \varphi(w_i).$$

Scaled dot-product attention (Problem 3):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad \text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

Tip: subtract $\max(\mathbf{x})$ before exponentiating for numerical stability.

Problem 1: Dictionary Sentiment, and Why Domain Matters

Snippet (10-K Risk Factors, stylised):

*“Demand remained **positive**, but **litigation** exposure and macroeconomic **uncertainty** could adversely affect results; management believes the outlook is **strong**.”*

Task A. Using Loughran–McDonald’s 6 categories (negative, positive, uncertainty, litigious, strong/weak modal), classify the four bold words. Which category does “litigation” / “uncertainty” fall into?

Task B. A *general* (Harvard Inquirer) dictionary tags “litigation,” “liability,” and “tax” as negative. The chapter notes $\sim 3/4$ of Harvard “negative” words are neutral/positive in filings. What does that imply for a naive negative-word count on legal-heavy filings?

Task C. Compute the net sentiment $(\#pos - \#neg)/\#total$ for the snippet (10 words after dropping stop-words), using a finance lexicon where only *negative* & *positive* categories count toward polarity.

Problem 2: Mean Pooling is Order-Blind

Two sentences with opposite meaning:

- A : “Revenue rose, costs fell.”
- B : “Costs rose, revenue fell.”

Task A. The two sentences contain the *same multiset of words*. Show that the mean BoW embedding $\bar{\varphi}(A) = \bar{\varphi}(B)$ exactly. Why?

Task B. A long-only equity signal goes *long* on positive-sentiment filings. If your representation cannot distinguish A from B , what failure mode appears on filings where the *subject* of “rose” vs. “fell” matters?

Task C. Name the architectural ingredient (from the lecture) that restores order sensitivity, and state in one line *why* it can connect “revenue” to “rose” across a long sentence where an RNN struggles.

Problem 3: A 3×3 Self-Attention Trace

Sequence: "risk", "increased", "materially". Head dimension $d_k = 2$.

$$Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}$$

Task A. Compute $S = QK^\top$ (entry $(i,j) = \mathbf{q}_i^\top \mathbf{k}_j$).

Task B. Scale: $\tilde{S} = S/\sqrt{2}$ (two decimals).

Task C. Softmax row-wise $\rightarrow A$; **Task D.** output $O = AV$.

Discussion: Which token does "risk" attend to most? If we set $d_k = 64$ with the same raw dot products, are the weights *sharper* or *flatter*?

Case Study: Dictionary vs. Embedding Sentiment

Goal: on a handful of SEC-style snippets, compare a **Loughran–McDonald-style dictionary** score against a **transformer embedding** similarity score, and see where they disagree.

Stack:

- `sentence_transformers` — contextual embeddings (SBERT).
- A small finance positive/negative word list — the dictionary baseline.

Notebook: `code/notebooks/16-ai-ml-finance-text/demo.ipynb`

Setup

```
pip install sentence-transformers
```

No API key needed — embeddings run locally on CPU.

Case Study: The Code

```
from sentence_transformers import SentenceTransformer, util

SNIPPETS = [
    "Revenue rose 12% and margins expanded.",          #
        clearly positive
    "Litigation exposure and uncertainty increased.",    #
        negative, legal-heavy
    "Costs fell, revenue fell, outlook unchanged.",      #
        order-sensitive
]

POS = {"rose", "expanded", "growth", "strong", "gain"}
NEG = {"fell", "litigation", "uncertainty", "loss", "decline"}

def dict_sentiment(text):
    w = text.lower().replace(",", " ").replace(".", " ").split()
    return (sum(t in POS for t in w) - sum(t in NEG for t in w)) / len(w)

model = SentenceTransformer("all-MiniLM-L6-v2")
```

Case Study: Diagnostic Questions

- Q1.** For each snippet, report the dictionary score and the embedding similarity. Where do the two *disagree* in sign or ranking?
- Q2.** Snippet 3 (“Costs fell, revenue fell. . .”) contains the *negative* word “fell” twice. Does the dictionary call it negative? Is that correct, given costs falling is *good*? What does this reveal about **context**-free scoring?
- Q3.** Replace “uncertainty” with “opportunity” in snippet 2. Does the dictionary score flip? Does the embedding similarity move *more* or *less* than the dictionary? Why?

Optional extension

Add “positive” once to snippet 2 (recall “positive” is *cautious* in credit-rating language). Does the dictionary over-credit it? Sketch how you would disambiguate using the document’s genre.

Discussion: Dictionary vs. Embedding vs. LLM

For each scenario, pick **dictionary**, **embedding/encoder**, or **LLM**, and justify on cost, auditability, label availability, and context.

- ① A regulator must *reproduce* a sentiment score from a published, auditable rule on 200k filings — transparency is mandatory.
- ② A desk wants to rank 50k earnings-call paragraphs by similarity to a “supply-chain risk” theme, with no labeled training data.
- ③ An analyst asks: “Summarize how this 10-K’s risk language changed versus last year, and flag any new litigation exposure.”

Frame your answer around

Transparency / reproducibility, context sensitivity, labeled-data needs, generation vs. scoring, cost, and the strategic production of the text.

Solution: Problem 1 — Dictionary Sentiment

Task A — categories: “positive” $\rightarrow$ *positive*; “strong” $\rightarrow$ *strong modal* (and positive-leaning); “litigation” $\rightarrow$ *litigious*; “uncertainty” $\rightarrow$ *uncertainty*. Note: litigious and uncertainty are *not* polarity-negative per se — they are their own categories.

Task B: A general dictionary tags “litigation” as negative, so a naive negative count **systematically over-penalises legal-heavy filings** — precisely the Loughran–McDonald finding ($\sim 3/4$ of Harvard negatives are neutral/positive in filings).

Task C: With only *positive/negative* counting toward polarity: positives = {positive, strong} = 2, negatives = 0 (litigious/uncertainty excluded). Net = $(2 - 0)/10 = +0.20$. A general dictionary would instead score this *negative* — the wrong sign.

Solution: Problem 2 — Order-Blindness

Task A: Mean pooling sums per-word embeddings then divides by n . Addition is commutative, so any permutation of the *same* words yields the *identical* mean. A and B share the multiset {revenue, rose, costs, fell} $\Rightarrow \bar{\varphi}(A) = \bar{\varphi}(B)$ exactly.

Task B: A sentiment-driven long-only signal cannot tell “revenue rose” (bullish) from “revenue fell” (bearish): it would take the *same* position on opposite fundamentals — a sign error that no amount of training fixes if the representation discards order.

Task C: Self-attention restores order sensitivity (with positional information). It can directly connect “revenue” to “rose” because every position attends to every other in one step — no sequential path to decay, unlike an RNN whose gradient vanishes over long spans.

Solution: Problem 3 — Attention Trace

Here $K = Q$, so $S = QQ^\top$:

$$S = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \quad \tilde{S} = S/\sqrt{2} \approx \begin{pmatrix} 0.71 & 0.71 & 0.00 \\ 0.71 & 1.41 & 0.71 \\ 0.00 & 0.71 & 0.71 \end{pmatrix}.$$

Row-wise softmax ($\approx$):

$$A \approx \begin{pmatrix} 0.422 & 0.422 & 0.156 \\ 0.270 & 0.461 & 0.270 \\ 0.156 & 0.422 & 0.422 \end{pmatrix}.$$

Output $O = AV$, with $V = [(2, 0), (0, 2), (1, 1)]^\top$:

$$O \approx \begin{pmatrix} 1.00 & 1.00 \\ 0.81 & 1.19 \\ 0.73 & 1.27 \end{pmatrix}.$$

Discussion: "risk" attends most to itself and "increased" (tied at 0.422). At $d_k = 64$ the same raw dot products are divided by a larger $\sqrt{d_k}$ *only if unscaled* — but with the same *pre-scaling* magnitudes the larger

Wrap-Up: Key Takeaways

From Problem 1: a *domain* lexicon (Loughran–McDonald) avoids the systematic negative bias a general dictionary inflicts on legal-heavy filings — the canonical motivation for finance-specific NLP.

From Problem 2: bag-of-words mean pooling is provably order-blind; opposite-meaning sentences collapse to one vector. Order matters in finance, and self-attention is the fix.

From Problem 3: attention is a weighted average of value vectors; weights are softmaxed scaled dot products; $\sqrt{d_k}$ keeps the softmax out of its saturated, vanishing-gradient regime.

From the Case Study: dictionary and embedding scores disagree exactly where *context* matters (“costs fell” is good; “positive” is cautious in ratings) — the case for moving up the ladder to encoders and LLMs.

Next practical

Building the transformer block end-to-end (LLM-foundations): multi-head attention, positional encoding, and a first fine-tuning pass.

APPENDIX

Stretch Problems

Extra / optional material — beyond the core lecture.

Stretch Problem A: tf-idf by Hand

Three one-sentence “documents”:

- d_1 : “revenue revenue growth”
- d_2 : “revenue decline”
- d_3 : “growth growth growth”

Task. With $\text{tf} = \text{raw count}$ and $\text{idf}(w) = \log \frac{N}{\text{df}(w)}$ ($N = 3$), compute the tf-idf weight of “revenue” in d_1 and of “growth” in d_3 . Which term is up-weighted, and why does tf-idf *down-weight* a word appearing in every document?

Point

tf-idf rewards rare, document-specific words — the BoW refinement that made 1990s filing analysis workable before embeddings existed.

Stretch Problem B: Reflexivity in a Backtest

A text-based “negative-sentiment $\Rightarrow$ short” signal backtests with a Sharpe of 1.4 over 2015–2019.

Task A. Name two reasons the live Sharpe may be far lower: (i) the signal becomes *public*; (ii) the strategy’s own trades move prices (reflexivity).

Task B. The chapter warns LLMs can *reconstruct historical series from parametric memory*, producing spurious predictability. If your sentiment model was pre-trained on data *overlapping* the backtest window, what bias contaminates the result, and how would you guard against it?

Point

AI systems in finance are *actors within* the market, not observers — look-ahead and reflexivity are first-order design concerns, not footnotes.